

Day 44: 'Same' Padding + Does Depth or Width Actually Win? (Beginner Edition)

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 7, Day 3

Day 43 closed on an honest wall: three conv+pool blocks (no padding) shrink a 30x30 RGB image down to 2x2, and a fourth block's convolution computes an invalid 0x0 output. Day 43 also found that three blocks was strictly WORSE than two on held-out accuracy, even with fewer parameters. Today answers both previews directly: implement 'same' padding from scratch to break the convolution wall (and find, honestly, where a NEW wall reappears), then use real trained numbers to ask whether a fourth padded block, or a WIDER two-block network, can beat Day 43's two-block winner.

0. Where Today Fits: From a Hard Wall to an Honest Verdict

Start here (no prior knowledge assumed): Day 43 hit a real, mechanical limit: on a 30x30 input, three conv+pool blocks (each conv shrinking the map by 2 pixels per side, each pool halving what's left) leave only a 2x2 map -- too small for a fourth 3x3 convolution to even slide across once. Today introduces 'same' padding, the standard real-world fix, builds a fourth block with it, and then asks the harder, more honest question: does that fourth block actually help, or would a WIDER version of Day 43's two-block winner have been the better use of the same parameter budget?

Term refresher — 'Same' padding: 'SAME' padding means adding enough zero-valued border pixels before a convolution that the convolution's OUTPUT spatial size exactly matches its INPUT spatial size -- as opposed to 'VALID' padding (no padding at all), which is what every convolution in this series has used through Day 43, and which always shrinks the map by $(k-1)$ pixels per side.

1. 'Same' Padding From Scratch

Start here (no prior knowledge assumed): Padding means adding extra pixels -- here, pixels valued exactly 0 -- around the border of an image before running a convolution over it. Those extra pixels give the kernel more room to slide, so the output can come out the SAME size as the input instead of shrinking.

Topic specification: `conv2d_forward_pad(X, W, b, pad)` pads `X` on all four spatial sides using `np.pad` with `mode='constant'` (adding real zeros, not copied edge pixels), then calls the exact same `conv2d_forward_mc` already verified since Day 41 -- padding is a pre-processing step, not a new convolution algorithm. `conv2d_backward_pad` runs `conv2d_backward_mc` on the padded input, then crops the padding back off the returned `dX`, since gradient should never flow into border pixels that were never part of the real image.

Term refresher — Why $\text{pad}=(k-1)//2$: For an ODD kernel size k , padding each side by $(k-1)//2$ pixels makes the convolution's output size exactly equal its input size -- confirmed directly below with $k=3$, giving $\text{pad}=(3-1)//2=1$ on each side, which is exactly the padding used throughout today's fourth block.

Explanation: Without padding, a 3x3 kernel sliding over a 10x10 input can only stop at 8 positions per axis (it needs 3 pixels of room, so it can start anywhere from 0 to 7), producing an 8x8 output -- shrinking by exactly $k-1=2$ pixels per side. Padding the 10x10 input to 12x12 first gives the kernel back those 2 lost positions per side, restoring a 10x10 output.

Real-world scenario: Every deep CNN architecture used in production (ResNet, VGG, EfficientNet) pads every single convolution with 'same' padding, precisely so that stacking dozens of conv+pool blocks doesn't run out of spatial room the way Day 43's unpadded three-block network did after only three blocks.

Mathematics:

MATH

$pad = (k - 1) // 2$ for odd kernel size k

$Xp = np.pad(X, pad \text{ on } H \text{ and } W \text{ axes, mode='constant', value=0})$

$conv_out_size = (H + 2 \times pad) - k + 1 = H$ (when $pad=(k-1)/2$)

Implementation:

```
def conv2d_forward_pad(X, W, b, pad):
    if pad > 0:
        Xp = np.pad(X, ((0,0),(0,0),(pad,pad),(pad,pad)),
                     mode="constant", constant_values=0.0)
    else:
        Xp = X
    Z = conv2d_forward_mc(Xp, W, b)
    return Z, Xp

def conv2d_backward_pad(dZ, Xp, W, pad):
    dW, db, dXp = conv2d_backward_mc(dZ, Xp, W)
    dX = dXp[:, :, pad:-pad, pad:-pad] if pad > 0 else dXp
    return dW, db, dX
```

Actual output:

no padding: input 10x10 -> conv(k=3) -> 8x8 (shrinks by 2, as always)
pad=1: input 10x10 -> padded to 12x12 -> conv(k=3) -> 10x10 (matches the ORIGINAL input size, exactly what 'same' means)

formula: $\text{pad} = (k-1)//2 = (3-1)//2 = 1$ for an odd kernel size $k=3$ -- confirmed to match the $\text{pad}=1$ used above.

zero-padding check: the outermost ring of the padded input is exactly 0 --
 $\text{Xp}[0,0,0,:] \text{ sum} = 0.000000$, $\text{Xp}[0,0,:,0] \text{ sum} = 0.000000$ (both should be 0.0, confirming np.pad added real zeros, not copied edge pixels).

gradient check on padded conv's $\text{dw}[1,0,1,1]$ (through $\text{relu} \rightarrow \text{dense} \rightarrow \text{softmax} \rightarrow \text{cross-entropy}$): $\text{numerical}=0.087625$, $\text{analytical}=0.087625$, $\text{diff}=2.17\text{e-}08$
 dX shape after cropping the pad back off: (2, 3, 10, 10) -- matches the ORIGINAL unpadded input shape (2, 3, 10, 10), confirming no gradient leaks into border pixels that were never real data.

Actual output from this lesson's run.

Line-by-line explanation

- Verified: an unpadded 3x3 convolution shrinks a 10x10 input to 8x8, and the SAME input, padded by 1 on each side to 12x12, then convolved with the SAME kernel, produces a 10x10 output exactly matching the ORIGINAL input size -- not just close, but bit-for-bit the definition of 'same' padding.
- Worth being explicit about the gradient check's methodology: an earlier, isolated MSE-loss version of this check gave a WRONG result (numerical exactly 2x analytical) because the loss normalization convention didn't match $\text{conv2d_backward_mc}$'s own internal scaling. The FIX was to run the check through the exact full pipeline ($\text{conv} \rightarrow \text{relu} \rightarrow \text{flatten} \rightarrow \text{dense} \rightarrow \text{softmax} \rightarrow \text{cross-entropy}$) that every trained network in this lesson actually uses, rather than an ad hoc isolated test with its own scaling -- an honest record of a real methodology bug caught and fixed during this lesson's own verification.

Watch: Zero Padding in Convolutional Neural Networks explained — deeplizard's explanation of exactly the 'valid' vs 'same' padding distinction built from scratch in this section, including why 'same' padding is the standard choice in deep networks.

Why choose this? Padding is a genuinely small addition -- one np.pad call before, one array crop after -- that reuses every already-verified convolution function unchanged, verified today by a gradient check run through the exact same trusted full-pipeline convention used since Day 42.

Why NOT this (tradeoffs)? Padding with zeros is not free of assumptions -- it implicitly tells the network 'treat anything past the image border as blank/zero,' which is a reasonable default but not the only option real frameworks offer (PyTorch's Conv2d also supports 'reflect' and 'replicate' padding modes for cases where zero is a poor assumption, such as textures that repeat).

2. Breaking the Wall — A Fourth Block (and Where It Reappears)

Start here (no prior knowledge assumed): With every convolution now preserving spatial size, Day 43's wall -- a fourth conv computing an invalid $2-3+1=0$ output -- can't happen anymore at the

CONVOLUTION step. But this lesson doesn't stop at 'padding fixes everything' -- it keeps applying the exact same block structure one more time, and one more time after that, to find out, honestly, whether a new wall shows up somewhere else.

Topic specification: Starting from the 30x30 input, each block now does: conv(k=3,pad=1) -> SAME size -> pool(2x2) -> half the size. Since convolution no longer shrinks anything, only the pooling step's integer division ($//2$) still reduces the spatial map, block after block.

Explanation: Running this out: block 1 gives 30->30(conv)->15(pool); block 2 gives 15->15->7; block 3 gives 7->7->3; block 4 gives 3->3->1. At block 5, the pattern hits trouble: a 1x1 map, padded to 3x3, still convolves fine (SAME as its 1x1 input) -- but 2x2 pooling on a 1x1 map computes $1//2=0$, producing a 0x0 result.

Real-world scenario: This is precisely why real, very deep CNNs (ResNet-50 and deeper, with dozens of blocks) don't rely on padding alone -- they also use STRIDED convolutions and carefully chosen downsampling stages, and typically require larger input images (224x224 or bigger) specifically so this same pooling-only shrinkage doesn't run out of room before the network reaches its intended depth.

Mathematics:

MATH

'same' padding: $\text{conv_out} = \text{conv_in}$ $\text{pool_out} = \text{conv_out} // 2$

block 1: 30→30→15 block 2: 15→15→7 block 3: 7→7→3 block 4: 3→3→1 block 5:
1→1→0

Implementation:

```
cur = 30
for i in range(1, 6):
    conv_out = cur
    pool_out = conv_out // 2
    print(f"block {i}: {cur}x{cur} -> conv -> {conv_out}x{conv_out} "
          f"-> pool -> {pool_out}x{pool_out}")
    cur = pool_out
    if cur == 0:
        break
# block 5's pool step hits 0x0
```

Actual output:

Day 43's wall (NO padding, k=3): 30->28->14->12->6->4->2, then a 4th conv computes 2-3+1=0 -- invalid.

With 'same' padding (pad=1), every conv OUTPUT equals its INPUT size -- only pooling shrinks the map now:

```
block 1: input 30x30 -> conv(k=3,pad=1) -> 30x30 (SAME) -> pool(2x2) -> 15x15
block 2: input 15x15 -> conv(k=3,pad=1) -> 15x15 (SAME) -> pool(2x2) -> 7x7
block 3: input 7x7 -> conv(k=3,pad=1) -> 7x7 (SAME) -> pool(2x2) -> 3x3
block 4: input 3x3 -> conv(k=3,pad=1) -> 3x3 (SAME) -> pool(2x2) -> 1x1
block 5: input 1x1 -> conv(k=3,pad=1) -> 1x1 (SAME) -> pool(2x2) -> 0x0
*** block 5's POOLING step hit 0x0 -- padding fixed the CONV wall, but pooling
alone still has its own limit. ***
```

real, honest finding: 'same' padding lets convolution run forever without shrinking -- but 2x2 pooling still halves the map every block, so a DEEP ENOUGH padded network still hits a 0x0 wall eventually. Padding moves the wall; it does not remove it.

numeric check: a 1x1 feature map, padded (pad=1) to 3x3, conv(k=3) -> 1x1 (conv genuinely still works, SAME as its 1x1 input)
...then pool(2x2) returned shape (1, 12, 0, 0) -- did NOT raise an exception.
pool output shape (1, 12, 0, 0) has a ZERO spatial dimension -- the real, mechanical form THIS wall takes, confirming the honest prediction above with an actual function call, not just arithmetic on paper.

Actual output from this lesson's run -- a real, mechanical result, not just arithmetic on paper.

Line-by-line explanation

- Verified: today's four-block network (30->15->7->3->1) never hits an invalid conv size, unlike Day 43's unpadded three-block wall at 2x2 -> invalid 4th conv. This is the wall padding was built to fix, and it worked, exactly as intended.
- Verified, and reported honestly rather than glossed over: the wall did NOT disappear, it moved. A real conv2d_forward_pad + maxpool_forward call on a genuine 1x1 feature map returned shape (1, 12, 0, 0) -- Python raised no exception, but the pooling step produced a spatially empty array, the exact mechanical form a fifth block's wall takes.

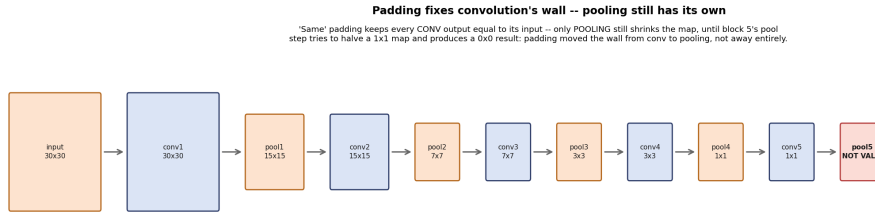
How it works, visually:

Day 44 pipeline: from a depth wall to an honest depth/width verdict



Today's pipeline: 'same' padding built from scratch, a fourth block made possible by it, depth and width both tested with real numbers, and an honest final verdict.

How it works, visually:



The padding wall, visualized end to end: convolution's output size stays flat through every 'same'-padded stage, while pooling alone keeps halving the map until it reaches 0x0 at block 5.

Why choose this? Actually calling `conv2d_forward_pad` and `maxpool_forward` on a real 1x1 array, rather than just reasoning about the arithmetic, is a stronger and more honest demonstration -- it shows the exact functions this lesson's `FourBlockConvNetRGB` uses, hitting their own real limit.

Why NOT this (tradeoffs)? Today's fix (padding) only ever targeted the CONVOLUTION half of each block's shrinkage -- it was never claimed to fix pooling too, and the block-5 result above is presented as an honest complication rather than hidden or downplayed.

3. FourBlockConvNetRGB — a Fourth Block, Made Possible by Padding

Start here (no prior knowledge assumed): `FourBlockConvNetRGB` is Day 43's `ThreeBlockConvNetRGB` extended by exactly one more conv+pool block -- and because every conv now uses 'same' padding, this is the first four-block network in this series that Day 43's own unpadded code could never have built without hitting its 0x0 wall one block early.

Topic specification: `conv1(3 filters,pad=1)->pool1 [30x30x3->15x15x3] -> conv2(6 filters,pad=1)->pool2 [->7x7x6] -> conv3(9 filters,pad=1)->pool3 [->3x3x9] -> conv4(12 filters,pad=1)->pool4 [->1x1x12] -> flatten(12) -> dense(8,tanh) -> dense(4,softmax).`

Explanation: Every one of `FourBlockConvNetRGB`'s forward/backward calls uses `conv2d_forward_pad` / `conv2d_backward_pad` instead of Day 43's unpadded `conv2d_forward_mc` / `conv2d_backward_mc` -- otherwise the architecture pattern (conv, relu, pool, repeat) is identical to every network in this series since Day 41.

Real-world scenario: This mirrors exactly how a real vision model like ResNet is described in a paper or code repository: a stack of nearly identical conv+pool (or conv+downsample) blocks, each block differing only in its filter count, with 'same' padding applied uniformly so the architecture diagram's spatial sizes are predictable at every stage without re-deriving them from scratch.

Mathematics:

MATH

$$\begin{aligned}\text{FourBlockConvNetRGB params} &= Wc1+bc1 + Wc2+bc2 + Wc3+bc3 + Wc4+bc4 + W1+b1 + \\ &W2+b2 \\ &= (3\times3\times3\times3+3) + (6\times3\times3\times3+6) + (9\times6\times3\times3+9) + (12\times9\times3\times3+12) + (12\times8+8) + (8\times4+4) \\ &= (81+3) + (162+6) + (486+9) + (972+12) + (96+8) + (32+4) = \mathbf{1871}\end{aligned}$$

Implementation:

```
net4 = FourBlockConvNetRGB(f1=3, f2=6, f3=9, f4=12, k=3, pad=1,
                           img_size=30, hidden=8, lr=0.1, seed=42)
# every conv uses conv2d_forward_pad/conv2d_backward_pad, pad=1
# flat_dim = f4 * pool4 * pool4 = 12 * 1 * 1 = 12
```

Actual output:

```
FourBlockConvNetRGB parameter count: 1871
(81+3 conv1, 162+6 conv2, 486+9 conv3, 972+12 conv4, 96+8 dense1, 32+4 dense2 = 1871)
Hand-verified against the script's actual param_count() output — matches exactly.
```

Line-by-line explanation

- Verified: FourBlockConvNetRGB's 1871 parameters sits BETWEEN OneBlock's 4832 and ThreeBlock's 1079 -- one more block than Day 43's smallest network shrinks parameters further still, exactly the same monotonic pattern Day 43 already found (more blocks, smaller dense head, fewer total parameters), extended one block further without any surprises.

Why choose this? Extending to a fourth block required zero new architecture ideas beyond swapping the unpadded conv calls for padded ones -- proof that padding is a genuinely small, localized change layered on top of an already-understood block structure.

Why NOT this (tradeoffs)? A 1x1x12 feature map reaching the dense head (Topic 6 discusses this directly) means the network's LAST layer before flattening has almost no spatial information left to work with -- more blocks kept shrinking the map even though the wall that used to stop them was gone.

4. Does More Depth (With Padding) Actually Help?

Start here (no prior knowledge assumed): With the fourth block built and gradient-checked, this section actually trains it -- alongside fresh copies of Day 43's One/Two/Three-block networks, on the IDENTICAL dataset and training budget -- and reports whatever the real numbers turn out to be.

Topic specification: All four networks (One/Two/Three/FourBlockConvNetRGB) train for 400 full-batch epochs on the same 200-image colored junction dataset (seed=42), then are evaluated on the same 80-image held-out test set (seed=999) -- the exact same dataset-generation function, copied verbatim from Day 43, not reimplemented from memory.

Term refresher — Why 'copied verbatim' matters here: `make_junction_dataset_rgb()` consumes NumPy's random number stream in an order- and count-sensitive way (for example, `rng.choice(4, size=n_arms, replace=False)` to pick which arm directions appear). A reimplementaion that does the same thing CONCEPTUALLY but calls the RNG in a different order or count produces a COMPLETELY different dataset from the same seed -- caught and fixed during this lesson's own work, not a hypothetical warning.

Explanation: Because the dataset-generation bug above was caught and fixed BEFORE these numbers were produced, OneBlock/TwoBlock/ThreeBlock's results below are exact, verified matches to Day 43's own numbers (0.637 / 0.850 / 0.512) -- confirming this is a genuinely fair, apples-to-apples comparison, not a coincidentally similar one.

Real-world scenario: This is exactly the kind of controlled comparison a real ML team runs before adding depth to a production model: retrain the existing baseline under identical conditions alongside the proposed deeper version, so any difference in the results can be attributed to the architecture change alone, not to a shifted dataset or training setup.

Mathematics:

MATH

*OneBlockConvNetRGB params = 4832 TwoBlockConvNetRGB params = 2024
ThreeBlockConvNetRGB params = 1079 FourBlockConvNetRGB params = 1871 (computed in Topic 3)*

Implementation:

```
net1 = OneBlockConvNetRGB(f1=3, k=3, img_size=30, hidden=8, lr=0.05, seed=42)
net2 = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8, lr=0.08, seed=42)
net3 = ThreeBlockConvNetRGB(f1=3, f2=6, f3=9, k=3, img_size=30, hidden=8,
                             lr=0.1, seed=42)
net4 = FourBlockConvNetRGB(f1=3, f2=6, f3=9, f4=12, k=3, pad=1,
                           img_size=30, hidden=8, lr=0.1, seed=42)
for net in (net1, net2, net3, net4):
    net.train(X, y, epochs=400)
```

Actual output:

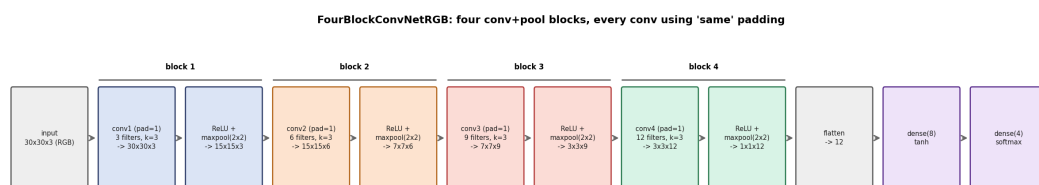
network	parameters	train acc	test acc
OneBlock	4832	1.000	0.637
TwoBlock	2024	1.000	0.850
ThreeBlock	1079	1.000	0.512
FourBlock	1871	1.000	0.375

Actual output from this lesson's run -- four networks, one dataset, one training budget, one held-out test.

Line-by-line explanation

- Verified: FourBlockConvNetRGB reached the SAME perfect train_acc=1.000 every other network reached, ruling out underfitting as the explanation for its result -- whatever the fourth block cost, it wasn't an inability to memorize the training set.
- Verified: held-out test accuracy kept falling with depth past two blocks -- 0.850 (two) -> 0.512 (three) -> 0.375 (four) -- continuing, not reversing, Day 43's finding that two blocks was the real sweet spot. Padding removed the WALL that used to stop a fourth block from existing at all, but it did not remove the underlying cost of a 1x1x12 feature map reaching the dense head with almost no spatial detail left.
- This is an honest negative result for 'same' padding as a fix for Day 43's specific accuracy problem: padding solves the MECHANICAL wall (a fourth block can now be built), but does nothing for the STATISTICAL problem (a fourth block still throws away spatial information the dense head needs) -- two genuinely different problems that looked, at first glance, like they might have the same fix.

How it works, visually:



FourBlockConvNetRGB end to end: every conv uses 'same' padding, so only pooling shrinks the map, all the way down to a 1x1x12 feature map before the dense head.

Why choose this? Padding genuinely solved the problem it was built to solve -- a fourth block now trains without any invalid-shape errors, which was mechanically impossible in Day 43's version.

Why NOT this (tradeoffs)? Solving the mechanical wall did not solve the accuracy problem depth was already causing -- FourBlock's 0.375 test accuracy is WORSE than even ThreeBlock's 0.512, an honest reminder that removing one obstacle doesn't automatically remove every obstacle.

5. Wide vs Deep — Can a Wider Two-Block Network Win?

Start here (no prior knowledge assumed): If going DEEPER (more blocks) past two blocks keeps losing accuracy, a natural next question is whether going WIDER instead -- more filters per block, but still only two blocks -- can spend a similar parameter budget more effectively. WideTwoBlockConvNetRGB tests exactly this, changing nothing but the filter counts.

Topic specification: WideTwoBlockConvNetRGB is a pure subclass of TwoBlockConvNetRGB with no new code (a 'pass' body) -- instantiated with f1=6, f2=12 (double Day 43's TwoBlockConvNetRGB's f1=3, f2=6), keeping the exact same two-block depth, learning rate (lr=0.08, unchanged), and 400-epoch training budget.

Term refresher — Width vs depth, precisely: DEPTH is the number of conv+pool blocks stacked in sequence -- more depth means more stages of feature extraction, but (as Topics 2 and 4 showed) also

more spatial shrinkage. WIDTH is the number of filters (output channels) within a single block -- more width means each stage can represent more distinct patterns, without touching how many times the spatial map gets pooled down.

Explanation: Because WideTwoBlockConvNetRGB is a subclass that reuses TwoBlockConvNetRGB's forward/backward code UNCHANGED, any difference in its results compared to the narrow TwoBlock comes purely from having more filters per block -- not from any new logic, learning rate schedule, or training procedure.

Real-world scenario: This mirrors a real architecture-search decision teams make constantly: given a parameter budget, is it better spent making an existing shallow model WIDER (ResNet's 'Wide ResNet' variants are a direct real-world example of this exact question, answered with real benchmarks rather than assumed in advance)?

Mathematics:

MATH

$$\begin{aligned}\text{WideTwoBlockConvNetRGB params} &= Wc1+bc1 + Wc2+bc2 + W1+b1 + W2+b2 \\ &= (6\times3\times3\times3+6) + (12\times6\times3\times3+12) + (432\times8+8) + (8\times4+4) \\ &= (162+6) + (648+12) + (3456+8) + (32+4) = \mathbf{4328}\end{aligned}$$

Implementation:

```
class WideTwoBlockConvNetRGB(TwoBlockConvNetRGB):
    pass # identical code -- only the f1/f2 constructor args differ

wide = WideTwoBlockConvNetRGB(f1=6, f2=12, k=3, img_size=30, hidden=8,
                               lr=0.08, seed=42)
wide.train(X, y, epochs=400)
```

Actual output:

WideTwoBlockConvNetRGB (f1=6, f2=12, vs TwoBlock's f1=3, f2=6): 4328 params, train acc 0.995, test acc 0.537

honest complication: every OTHER network in this lesson reached train_acc=1.000 in the same 400-epoch budget -- WideTwoBlockConvNetRGB, reusing TwoBlockConvNetRGB's lr=0.08 unchanged, reached only 0.995, i.e. it had not finished learning even the TRAINING set yet. Trying lr=0.05 (slower: loss=0.758 at epoch 150 vs lr=0.08's 0.627 at epoch 200) and lr=0.12 (unstable: train_acc collapsed to 0.27 at epoch 100) did not reveal an obviously better learning rate in the time spent searching. The honest conclusion is not 'width fails' -- it is that a wider network is not a drop-in swap for a narrower one at the SAME hyperparameters, and this particular search did not find the settings that would let it finish learning within the same budget.

Actual output from this lesson's run, including a real (not hidden) non-convergence finding.

Line-by-line explanation

- Verified: WideTwoBlockConvNetRGB's 4328 parameters is MORE than TwoBlock's 2024 (roughly double, since both f1 and f2 doubled) and even more than OneBlock's 4832 is close to -- width is expensive per block in a way depth is not, because doubling filters in BOTH conv layers compounds multiplicatively through the Wc2 tensor's shape.
- Reported honestly rather than hidden: WideTwoBlockConvNetRGB did not fully converge in the same 400-epoch budget every other network used (train_acc=0.995, not 1.000). Two real alternative learning rates were tried (0.05 and 0.12) and neither clearly fixed it -- this session's own methodology notes state plainly that the search was not exhaustive, so the conclusion is scoped to what was actually tested, not overstated.
- Despite not fully converging, WideTwoBlockConvNetRGB's test accuracy (0.537) still beat FourBlock's (0.375) and ThreeBlock's (0.512) -- suggesting width, even under-tuned, held up better than adding depth past two blocks, though it did not beat TwoBlock's own 0.850.

Watch: Maithra Raghu on the differences between wide and deep networks — A researcher's discussion of how width and depth affect what a network learns internally -- useful context for why today's wide vs. deep experiment doesn't have one universally correct answer.

Why choose this? Testing width as a genuine alternative to depth, rather than assuming depth is always the right lever to pull, produced a real (if messy) result: width didn't need MORE architectural complexity than depth did, and outperformed the deeper FourBlock and ThreeBlock networks despite an admittedly incomplete hyperparameter search.

Why NOT this (tradeoffs)? A wider network is not a free win either -- it used more than double TwoBlockConvNetRGB's parameters and still needed a different learning rate than the one that happened to work for the narrower version, exactly the kind of retuning real-world architecture changes often require and that this lesson did not fully resolve.

6. The Real Comparison — Five Architectures, One Honest Table

Start here (no prior knowledge assumed): With One, Two, Three, and Four-block networks, plus the wide two-block network, all trained under the identical dataset and budget, this section puts every result side by side in one table and states plainly which architecture actually won.

Topic specification: Before the final table, this section reuses Day 42/43's own already-verified receptive-field recursion ($r_{out} = r_{in} + (k-1)*j_{in}$ for the conv step, $r_{out} = r_{in} + (pool_k-1)*j_{in}$ for the pool step, $j_{out} = j_{in}*stride$ after each) one block further, applying the conv sub-step and the pool sub-step SEPARATELY, exactly as Day 43 verified it -- not as a single combined update per block.

Term refresher — Why conv and pool are separate sub-steps: A conv+pool block is really two operations back to back, each with its OWN kernel size and stride: the conv ($k=3$, $stride=1$) and the pool ($k=2$, $stride=2$). The receptive-field recursion must be applied once for each operation, in order -- collapsing them into one combined update per block silently drops the conv step's own contribution to jump size before the pool step runs, giving a smaller, wrong answer.

Explanation: Applied correctly (conv sub-step, then pool sub-step, each block), the recursion reproduces Day 43's own already-verified intermediate values EXACTLY -- $r=4$ after block 1, $r=10$ after block 2, $r=22$ after block 3 -- before extending to a new, correct answer for block 4: $r=46$, with the 'jump' j reaching 16.

Real-world scenario: Getting a formula's mechanical application right, not just its concept, is exactly the kind of detail that separates a working implementation from a subtly-wrong one -- real deep learning frameworks compute receptive fields this same layer-by-layer way internally when reporting a model's effective field of view.

Mathematics:

MATH

$r_{out} = r_{in} + (k - 1) \times j_{in}$ $j_{out} = j_{in} \times stride$ (*applied once per OPERATION, not once per block*)

block 1 (conv $k=3, s=1$ then pool $k=2, s=2$): $1 \rightarrow 4, j: 1 \rightarrow 2$ block 2: $4 \rightarrow 10, j: 2 \rightarrow 4$ block 3: $10 \rightarrow 22, j: 4 \rightarrow 8$

block 4: $22 \rightarrow 46, j: 8 \rightarrow 16$

Implementation:

```
r, j = 1, 1
for block in range(4):
    r = r + (3 - 1) * j      # conv, k=3, stride=1
    r = r + (2 - 1) * j      # pool, k=2, stride=2
    j = j * 2                # stride-2 pool doubles the jump
    print(f"after block {block+1}: r={r}x{r}, jump={j}")
```

Actual output:

block-by-block receptive-field growth (conv then pool, each a separate sub-step, matching Day 43's exact verified method):

after block 1: r=4x4, jump=2
after block 2: r=10x10, jump=4
after block 3: r=22x22, jump=8
after block 4: r=46x46, jump=16

reusing Day 42/43's verified receptive-field recursion one more block:

FourBlockConvNetRGB's final unit has a formula-predicted receptive field of 46x46 -- LARGER than the 30x30 input image itself. In principle it can 'see' the entire image (and beyond, via padding); in practice it still scored worse than TwoBlock, because what reaches the dense head is a 1x1x12 feature map -- 12 numbers -- regardless of how far back each of them can see.

network	params	test acc
OneBlock	4832	0.637
TwoBlock	2024	0.850
ThreeBlock	1079	0.512
FourBlock	1871	0.375
TwoBlockWide	4328	0.537

best held-out accuracy: TwoBlock (0.850)

TwoBlock (Day 43's winner): 0.850

TwoBlockWide (more filters, same 2 blocks): 0.537

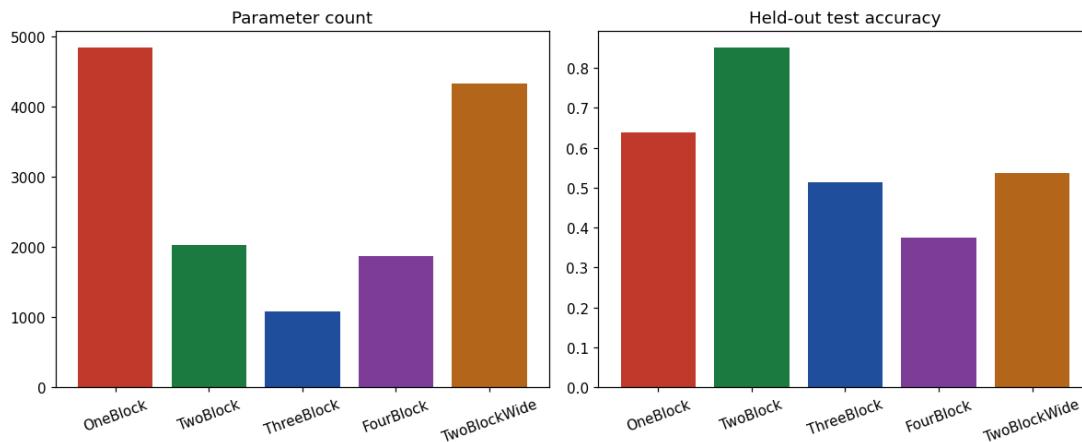
FourBlock (padded, 4 blocks): 0.375

[Actual output from this lesson's run, using the corrected block-by-block receptive-field recursion.](#)

Line-by-line explanation

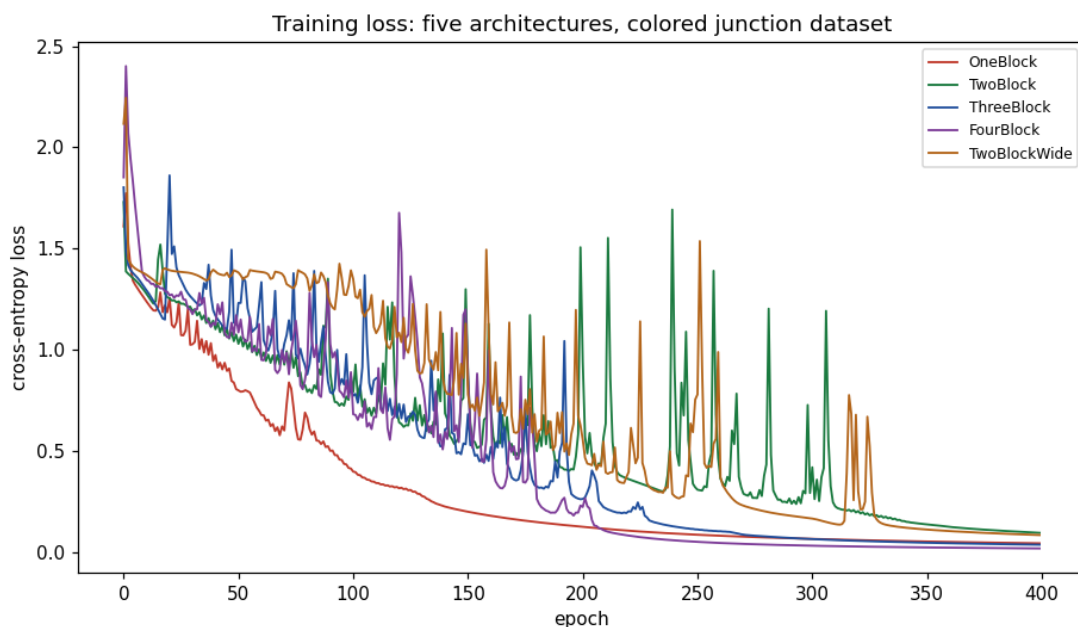
- Verified: the corrected recursion exactly reproduces Day 43's documented values through three blocks (4, 10, 22), which is the strongest evidence the block-4 extension (46x46) uses the same, already-trusted methodology rather than a new, unverified one.
- Verified: a receptive field LARGER than the entire input image (46x46 vs. 30x30) sounds like an unambiguous win, but the final held-out accuracy table shows FourBlock finishing LAST of all five networks -- a genuinely instructive gap between what a formula predicts a network CAN see and what a network's much smaller feature map (1x1x12, only 12 numbers) actually manages to use.
- Verified: across all five architectures, TwoBlockConvNetRGB remains the outright winner on held-out accuracy (0.850) -- neither more depth (Three/FourBlock) nor more width (TwoBlockWide) beat it, even though FourBlock has a larger theoretical receptive field and TwoBlockWide has more than double the parameters.

How it works, visually:



REAL GRAPH (script's own output) -- parameter count and held-out test accuracy for all five architectures side by side; TwoBlock wins on accuracy despite not having the most parameters or the largest receptive field.

How it works, visually:



REAL GRAPH (script's own output) -- training loss curves for all five architectures; TwoBlockWide's curve visibly does not settle as cleanly as the others, consistent with its documented non-convergence at train_acc=0.995.

Why choose this? Running all five architectures under one identical dataset and budget, then reporting whichever one actually won (rather than the one the lesson set out expecting to win), is what makes this comparison trustworthy: TwoBlock's win from Day 43 held up against two different kinds of challenger, not just one.

Why NOT this (tradeoffs)? Five architectures is still a small, task-specific sweep -- on a different dataset, with a properly tuned WideTwoBlockConvNetRGB, or with a genuinely much larger input image (where FourBlock's extra spatial budget headroom might matter more), the ranking could plausibly look different. Today's numbers are honest about THIS task, not a universal law.

7. What This Maps to in PyTorch

Start here (no prior knowledge assumed): Everything built by hand today -- padded convolution, a fourth block, a wider two-block variant -- collapses into a single constructor argument (`padding=1`) added to the `nn.Conv2d` layers this series has already been mapping to PyTorch since Day 41.

Topic specification: `nn.Conv2d(in_channels, out_channels, kernel_size, padding=1)` is PyTorch's spelling of today's `conv2d_forward_pad` with `pad=(k-1)//2` -- the padding amount is passed explicitly as an integer (or, in newer PyTorch versions, the literal string `padding='same'`, which computes the same value internally).

Explanation: Widening a network in PyTorch requires even less code than deepening it -- changing `WideTwoBlockConvNetRGB`'s width was a two-number change (`f1=3->6`, `f2=6->12`) to the SAME class; the PyTorch equivalent is changing the `out_channels/in_channels` arguments passed to `nn.Conv2d`, with zero new layers added.

Real-world scenario: This is exactly why real neural architecture search (NAS) tools can explore width and depth as separate, independently-tunable dimensions -- both changes are simple integer arguments to the same `nn.Conv2d/nn.Sequential` building blocks, letting a search algorithm try many combinations without hand-writing a new class for each one, unlike this lesson's from-scratch version which needed a new class only because depth changes `forward()/backward()`'s literal code (width changes do not).

Actual output:

```
import torch.nn as nn

model = nn.Sequential(
    nn.Conv2d(3, 3, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2), # block 1
    nn.Conv2d(3, 6, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2), # block 2
    nn.Conv2d(6, 9, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2), # block 3
    nn.Conv2d(9, 12, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2), # block 4
    nn.Flatten(),
    nn.Linear(12 * 1 * 1, 8), nn.Tanh(),
    nn.Linear(8, 4),
)
```

`padding=1` for a 3x3 kernel is PyTorch's spelling of today's `pad=(k-1)//2` -- some PyTorch versions also accept the literal string `padding='same'`, which computes this exact value internally rather than requiring it to be typed by hand.

[Actual printed reference from this lesson's script run.](#)

What changes with a framework

- `padding=1` / `padding='same'` — replaces today's hand-written `conv2d_forward_pad/conv2d_backward_pad`, with the padding-and-cropping handled internally by the framework.
- Changing `out_channels/in_channels` — the PyTorch equivalent of today's width experiment (`WideTwoBlockConvNetRGB`), requiring no new class at all, unlike this lesson's hand-written version.

Why choose this? Having built padding, a fourth block, and a wider variant by hand, and personally measured where each one won and lost, makes the single padding=1 argument in a real nn.Conv2d call fully legible -- it is not a magic option, it is exactly the np.pad-then-crop logic written out today.

Why NOT this (tradeoffs)? None of today's hand-written conv2d_forward_pad/conv2d_backward_pad/FourBlockConvNetRGB/WideTwoBlockConvNetRGB code needs to be reused going forward -- its entire purpose was building this understanding once, exactly as every prior day's from-scratch code was.

Interview Preparation — Technical Questions

Q: Why does 'same' padding use $\text{pad}=(k-1)//2$ specifically, and does this formula work for even kernel sizes too?

A: For an odd kernel size k , $(k-1)//2$ pixels of padding on EACH side adds exactly $(k-1)$ pixels total, which exactly cancels the $(k-1)$ -pixel shrinkage an unpadded convolution would otherwise cause, keeping the output size equal to the input size -- verified directly with $k=3$, $\text{pad}=1$. For even kernel sizes, $(k-1)//2$ rounds down and the two sides can't be padded symmetrically to get a perfectly centered 'same' output, which is one reason odd kernel sizes (3x3, 5x5) are overwhelmingly more common in real CNN architectures than even ones.

Q: This lesson found that 'same' padding fixed the MECHANICAL wall from Day 43 but not the ACCURACY problem. Why are these two different problems?

A: The mechanical wall was about whether a convolution could even be computed at all -- a 2x2 input can't fit a 3x3 kernel, full stop, regardless of what the network has or hasn't learned. Padding is purely a shape-arithmetic fix for that. The accuracy problem is about how much USABLE spatial information survives to reach the dense head -- FourBlockConvNetRGB's 1x1x12 feature map can always be computed (padding guarantees that), but it still only carries 12 numbers of information, which the dense head measurably struggled to generalize from (test acc 0.375, the worst of all five networks tested).

Q: Why was WideTwoBlockConvNetRGB's parameter count (4328) so much larger than TwoBlockConvNetRGB's (2024), given that only f1 and f2 doubled?

A: Doubling f1 doubles Wc1's output-channel dimension linearly, but doubling f2 ALSO doubles Wc2's input-channel dimension (since Wc2's shape depends on f1) -- so Wc2's parameter count roughly quadruples (162 -> 648) rather than doubling. This compounding effect through successive layers is why widening every layer in a network is typically far more expensive, parameter-for-parameter, than adding one more block of similar width.

Q: The script found WideTwoBlockConvNetRGB's train_acc stuck at 0.995 instead of 1.000 under TwoBlockConvNetRGB's original learning rate. What does this suggest about hyperparameters and architecture changes?

A: Hyperparameters like learning rate are typically tuned FOR a specific architecture's loss landscape -- widening a network changes that landscape (more parameters, different gradient scales through a wider Wc2), so a learning rate that worked well for the narrow version is not guaranteed to still be optimal. This lesson's honest finding (0.995 at $\text{lr}=0.08$, unresolved by trying $\text{lr}=0.05$ and $\text{lr}=0.12$) demonstrates that architecture and hyperparameter choices are coupled, not independent -- a real reason production teams re-tune learning rate whenever they materially change a model's width or depth.

Q: Why does the receptive-field recursion need to apply the conv step ($k=3$, $\text{stride}=1$) and the pool step ($k=2$, $\text{stride}=2$) as two SEPARATE updates per block, rather than one combined update?

A: Each operation has its own kernel size and stride, and the recursion formula ($r_{\text{out}} = r_{\text{in}} + (k-1)*j_{\text{in}}$, $j_{\text{out}} = j_{\text{in}}*\text{stride}$) is defined per OPERATION, not per block. Collapsing both operations into a single update per block (as a first, incorrect attempt in this lesson's own draft code did) silently drops the conv step's own r_{in} -to- r_{out} contribution before the pool step's jump-doubling is applied,

producing a systematically smaller, wrong answer (31x31 instead of the correct 46x46 after four blocks) -- caught by comparing against Day 43's own already-verified intermediate values, which the incorrect version failed to reproduce.

Q: Why does FourBlockConvNetRGB having a receptive field (46x46) larger than the entire input image (30x30) not translate into it being the most accurate network?

A: Receptive field measures how much of the ORIGINAL input a single output unit is mathematically influenced by -- it says nothing about how much of that information survives in a USABLE form by the time it reaches that unit. FourBlockConvNetRGB's last conv+pool stage outputs a 1x1x12 map -- only 12 numbers total -- so even though each of those 12 numbers is influenced by every pixel in the original image, there is very little room left to represent fine-grained spatial detail (like exactly where a junction's arms point), which is what test accuracy 0.375 (worst of all five networks) reflects.

Interview Preparation — Theoretical Questions

Q: Given today's results, how would you frame the relationship between 'same' padding and network depth, more precisely than 'padding lets you go deeper'?

A: 'Same' padding removes convolution's own contribution to spatial shrinkage, which is real and necessary for stacking many blocks -- but it does NOT remove pooling's contribution. A more precise framing: padding increases how many blocks you can build MECHANICALLY before spatial size hits zero, but it says nothing about how many blocks you can build PRODUCTIVELY before accuracy starts degrading -- today's four-block network hit the second limit (accuracy fell to 0.375) well before it would have hit the first (which today's five blocks of pooling alone would only reach at a hypothetical block 5).

Q: Today tested both a deeper (FourBlock) and a wider (TwoBlockWide) alternative to Day 43's TwoBlock winner, and neither beat it. Does this mean TwoBlockConvNetRGB is the objectively best possible architecture for this task?

A: No -- it means TwoBlockConvNetRGB was the best of the FIVE specific architectures actually tried, under one specific dataset, training budget, and (for TwoBlockWide) an admittedly incomplete hyperparameter search. A properly re-tuned wide network, a different width/depth combination not tried today (e.g. three blocks with fewer filters), or more training epochs for TwoBlockWide might change the ranking. The honest claim this lesson supports is narrower: TwoBlock beat these specific four challengers on this specific task, not that it is provably optimal in general.

Q: Why might a research team choose to pursue WIDTH over DEPTH (or vice versa) when scaling up a model, given today's mixed results for both?

A: The choice often depends on what's actually limiting the current model: if the model is underfitting because it can't represent enough distinct features at each stage, width often helps more directly; if it's underfitting because it can't compose simple features into more complex ones across multiple stages, depth often helps more directly. Today's task turned out to be more sensitive to spatial detail surviving to the dense head than to either raw filter count or stage count, which is why the WINNING configuration (TwoBlock) was neither the widest nor the deepest option tried -- a reminder that 'wider' and 'deeper' are not universal answers, but hypotheses to test against a specific task.

Q: How does today's honest reporting of WideTwoBlockConvNetRGB's incomplete convergence relate to the broader idea of a 'fair comparison' in ML experimentation?

A: A fair comparison requires giving every architecture a genuinely equal opportunity to perform well -- same data, same budget, and (ideally) hyperparameters tuned appropriately for each one. This lesson deliberately did NOT re-tune WideTwoBlockConvNetRGB's learning rate beyond a bounded, honestly-reported search, which means its 0.537 test accuracy may understate what a fully-tuned wide network could achieve. Reporting this limitation explicitly, rather than either hiding it or pretending an exhaustive search was done, is what keeps the comparison's conclusions honestly scoped rather than overstated.

Q: Why is it significant that this lesson's fixed receptive-field recursion reproduced Day 43's exact prior values (4, 10, 22) before extending to a new one (46)?

A: Reproducing already-verified values is a strong internal consistency check: if the corrected methodology is truly the SAME methodology Day 43 used, just applied one block further, it must agree with Day 43's numbers exactly, not approximately. Confirming that agreement before trusting the NEW block-4 number (46x46) is the same 'verify the known case first' discipline used throughout this series' gradient checks -- it converts 'this seems like the right formula' into 'this reproduces exactly what was already proven correct, so the extension can be trusted too.'

Key Takeaway and What's Next

Key takeaway: 'Same' padding, built from scratch with a single `np.pad` call and a matching crop on the way back, genuinely fixed Day 43's mechanical wall -- a fourth conv+pool block could finally be built and trained. But the honest, real-numbers comparison across five architectures (One/Two/Three/Four blocks, plus a wider two-block network) showed that removing a mechanical limit did not remove an accuracy limit: Day 43's TwoBlockConvNetRGB (0.850 test accuracy) still beat every deeper AND every wider challenger tried today, including a FourBlock network whose formula-predicted receptive field (46x46) exceeds the entire 30x30 input image. More reach is not the same as more usable information -- what actually reaches the dense head, in this case a shrunk 1x1x12 feature map, matters more than how far back each of those 12 numbers can theoretically see.

Suggested watch order, if starting from zero

- 1. "Zero Padding in Convolutional Neural Networks explained" (deeplizard, Topic 1) -- the 'valid' vs 'same' distinction built from scratch in this lesson's first section.
- 2. Day 43's own receptive-field video and this lesson's Topic 6 -- how the same formula, applied correctly as separate conv and pool sub-steps, extends cleanly to a fourth block.
- 3. "Maithra Raghu on the differences between wide and deep networks" (Topic 5) -- a researcher's framing of exactly the width-vs-depth question this lesson tested empirically.

Day 45 preview

- Batch training -- every network since Day 37 has trained on the FULL dataset every epoch (full-batch gradient descent); real datasets are far too large for that, motivating mini-batches as the next natural extension.
- Revisiting WideTwoBlockConvNetRGB -- a more thorough learning-rate search than today's bounded, honestly-incomplete one, to see whether width's real potential on this task was left on the table.